

File System Project Writeup

CSC 415 File System Project, Team Epsilon, Spring 2026

Repository: <https://github.com/CSC415-2026-Spring/csc415-filesystem-Edsonsb>

Name	Student ID	GitHub Name
Edson Sanchez Bernal	924372147	Edsonsb
Fnu Atiksha	923641508	atiksha05
Ahmed Rashad	924143880	AhmedRashad5627
Junxi Lin	923696927	Junxi-Lin

1. Submission Checklist

Readme Requirement	Where Addressed
Team roster, student IDs, GitHub names, and repository link	Included on cover page
Description of the file system	Sections 2 through 6
Table of shell commands that work, do not work, and limitations	Section 10
Screenshots showing required shell commands	Section 12
Detailed contribution table down to function level	Section 11

2. Description of the File System

This file system is a block-based design with persistent on-disk metadata, bitmap-based free-space management, a root directory initialized at format time, path-aware directory traversal, buffered file I/O, and shell command integration. The implementation separates low-level layout decisions from directory and file operations so that higher-level modules can rely on stable metadata helpers.

Component	Value / Layout	Purpose
Block size	512 bytes	Used consistently for the VCB, bitmap, directories, metadata, and file data.
Total blocks	19,531	Stored in the VCB and confirmed through the VCB dump.
VCB location	Logical block 0	Contains magic number, version, volume size, block size, bitmap location, and root directory location.
Free-space bitmap	Starts at block 1; uses 5 blocks	One bit per logical block. Bit value 1 means allocated; bit value 0 means free.
Root directory	Starts at block 6; uses 10 blocks	Initialized with 53 entries and the special entries . and ...

Component	Value / Layout	Purpose
Data region	Begins after root directory blocks	Used for file and directory data allocated through the bitmap helpers.

High-level structure. The VCB identifies whether the volume has been formatted. The bitmap records allocated and free blocks. Directory entries store names and file metadata on disk. File operations use an FCB table to track open files, offsets, access flags, and the corresponding directory entry.

3. Volume Control Block (VCB)

The Volume Control Block is the primary metadata block for the file system. It is stored at logical block 0 and is the first block checked during initialization. During mount, the file system reads the VCB and verifies the magic number. If the magic number matches MFSVCB1, the volume is treated as previously formatted. Otherwise, the file system formats the volume by writing the VCB, initializing the bitmap, and creating the root directory.

VCB Field	Stored Value	Meaning
Magic number	MFSVCB1	Identifies a formatted volume.
Version	1	Allows future layout changes to be distinguished.
Total blocks	19,531	Full logical block count in the volume.
Block size	512	Logical block size in bytes.
Bitmap start	1	First block of the free-space bitmap.
Bitmap size	5 blocks	Number of blocks reserved for the bitmap.
Root directory start	6	First block of the root directory.
Root directory size	10 blocks	Root directory storage area.
Root directory entries	53	Number of entries available in the root directory layout.

The VCB dump begins with the bytes 4D 46 53 56 43 42 31 00, which decode to MFSVCB1 followed by a null byte. The next values record the version, total block count, block size, free-space bitmap location, root directory location, and root directory entry count. The remaining bytes in the 512-byte block are unused and remain zero-filled.



Figure 1. VCB hexdump showing MFSVCB1, version, block count, block size, bitmap layout, and root directory layout.

4. Free-Space Bitmap

Free space is tracked with a bitmap. Each bit represents one logical block. A bit value of 1 means the block is allocated, and a bit value of 0 means the block is free. This keeps the free-space structure compact and makes allocation, release, and persistence straightforward.

Bitmap Item	Description
Allocation unit	One bit per block
Allocated block marker	1
Free block marker	0
Initial allocated range	Blocks 0 through 15
Reason first 16 blocks are used	VCB block 0, bitmap blocks 1-5, and root directory blocks 6-15
Persistence behavior	Bitmap is loaded during initialization, modified in memory, and written back

Bitmap Item	Description
	when allocation state changes or the file system unmounts.

The free-space dump begins with FF FF 00 00. Each FF byte has all eight bits set, so the first two FF bytes mark the first 16 blocks as allocated. The remaining zero bytes indicate blocks that are still free for future allocation.

```

student@student: ~/Desktop/csc415-filesystem-Edsonsb — less volume_dump.txt
~/Desktop/csc415-filesystem-Edsonsb
000300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000310: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000320: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000330: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000340: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000350: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000360: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000370: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000380: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000390: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0003A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0003B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0003C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0003D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0003E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0003F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000400: FF FF 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ??*****
000410: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000420: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000430: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000440: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000450: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000460: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000470: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000480: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000490: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0004A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0004B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0004C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0004D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0004E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
0004F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000500: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000510: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****
000520: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | *****

```

Figure 2. Free-space bitmap dump. The FF FF prefix marks the first 16 logical blocks as allocated.

5. Directory System and Path Resolution

The directory system begins with the root directory. The root directory is stored in blocks allocated by the bitmap and contains an array of permanent on-disk DirectoryEntry records. Each active entry records the name, whether the entry is in use, whether it is a directory, file size, starting block location, and timestamps/metadata. The design avoids runtime pointers in on-disk structures so entries can be safely written to disk and read again after remount.

At initialization, every root directory entry is cleared. Entry 0 is set to . and entry 1 is set to ... Because the root directory has no separate parent, both entries point back to the root directory. This makes later path traversal logic consistent for normal directories and for the root.

Directory Function / Helper	Implementation Purpose	
fs_mkdir	Creates a new directory entry and initializes its . and .. records.	
fs_rmdir	Removes a directory only after validation, including the empty-directory check.	
fs_opendir	Loads a directory and prepares iteration state.	
fs_readdir	Returns directory entries one at a time using fdDir state.	
fs_closedir	Releases directory iteration state.	
fs_getcwd	Returns the current working directory path.	
fs_setcwd	Changes the current working directory after resolving and validating a path.	
fs_isDir	Checks whether a resolved path refers to a directory.	
fs_resolvePath	Shared lookup helper for absolute paths, relative paths, ., and ...	
fs_resolveParentPath	Shared helper for locating a parent directory and final child name during create/delete/move operations.	

Path traversal supports absolute paths, relative paths, the current-directory token ., and the parent-directory token ... The fdDir structure was updated so directory iteration actually tracks loaded entries, current index, directory location, block count, and the fs_direntinfo record returned to the shell.

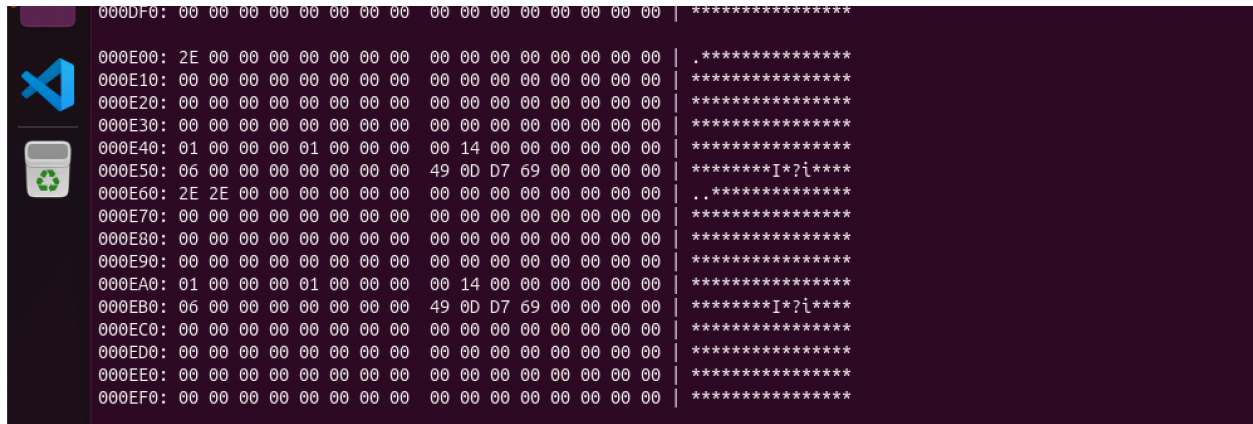


Figure 3. Root directory dump showing the . and .. directory entries and their metadata.

6. File Operations and Buffered I/O

The file I/O layer uses `b_io.c` for buffered reads, writes, seeks, opens, and closes. It bridges the directory/path-resolution layer and the block storage layer. The implementation maintains a File Control Block table so multiple files can be open at the same time while preserving independent file offsets and access flags.

FCB Field	Purpose
DirectoryEntry metadata	Stores the file metadata currently associated with the open descriptor.
Current file offset	Tracks where the next read or write begins.
Access flags	Stores <code>O_RDONLY</code> , <code>O_WRONLY</code> , <code>O_RDWR</code> , <code>O_CREAT</code> , <code>O_TRUNC</code> , and <code>O_APPEND</code> behavior.
Parent directory location	Identifies where the file entry must be updated.
Entry index	Identifies the exact directory slot for metadata updates.
File Function	Behavior
<code>b_open</code>	Resolves the path, validates file-vs-directory state, creates files when <code>O_CREAT</code> is present, applies <code>O_TRUNC</code> , applies <code>O_APPEND</code> offset behavior, and reserves an FCB slot.

FCB Field	Purpose
b_read	Reads from the current offset up to the requested count or EOF and advances the offset.
b_write	Writes from the current offset, expands the file when needed, handles block-boundary writes, updates size and modify time, and advances the offset.
b_seek	Moves the current offset within the existing file size. Sparse files are not supported, so seeking beyond EOF returns an error.
b_close	Releases the FCB slot for the open file. File metadata is updated during operations such as write, truncate, read timestamp update, and delete rather than being deferred only to close.
fs_isFile	Resolves a path and returns true only when the target is a regular file.
fs_delete	Resolves the file, frees its allocated data blocks, clears the directory entry, and persists the parent directory update.
fs_stat	Reports file metadata such as file size and block usage.

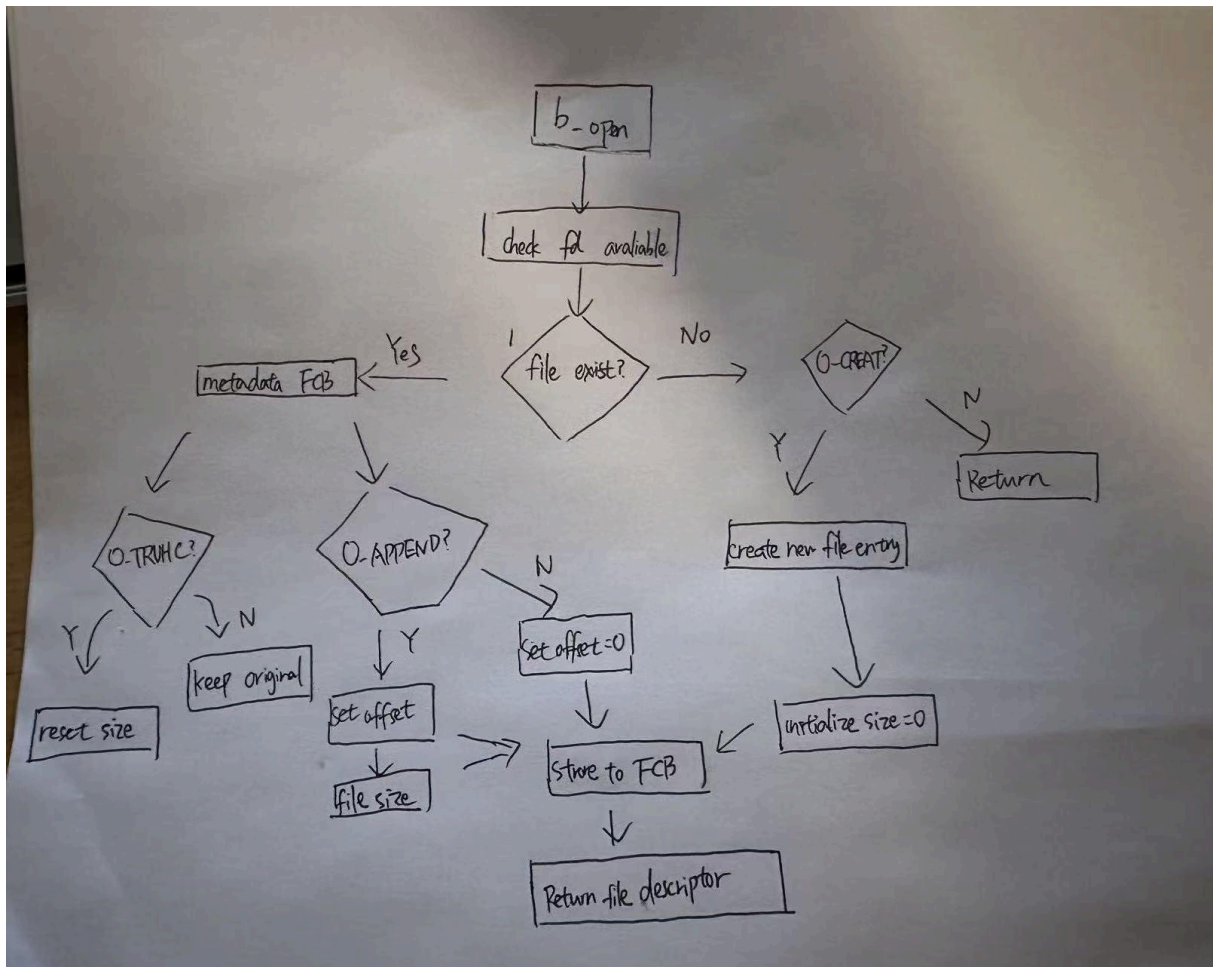


Figure 4. Design sketch for `b_open` logic: resolve file, handle `O_CREAT`, `O_TRUNC`, `O_APPEND`, store metadata in the FCB, and return a descriptor.

6.1 Representative File-I/O Logic

The following snippets summarize the main implementation decisions documented for the file layer.

Opening and creating files. `b_open` first resolves the path. If the file exists, its metadata is loaded into the FCB. If it does not exist and `O_CREAT` is present, the parent path is resolved and a new directory entry is created.

```

if (fs_resolvePath(filename, &entry, &parentLocation, &entryIndex) == 0)
{
    fcbArray[fd].entry = entry;
    fcbArray[fd].parentLocation = parentLocation;
}
  
```

```

    fcbArray[fd].entryIndex = entryIndex;
}
else if (flags & O_CREAT)
{
    fs_resolveParentPath(filename, &parentLocation,
        childName, sizeof(childName));
    /* create new directory entry */
}

```

Truncation. O_TRUNC must clear the file and release its previously allocated blocks; simply setting size to zero would leak space.

```

if ((flags & O_TRUNC) && flagsAllowWrite(flags))
{
    fs_freeBlocks(entry.location, oldBlocks);
    entry.size = 0;
    entry.location = 0;
    entry.timestamp = time(NULL);
}

```

Append mode. O_APPEND forces each write to begin at the current end of the file instead of the previous offset.

```

if (flags & O_APPEND)
{
    fcbArray[fd].offset = (off_t)fcbArray[fd].entry.size;
}

```

Multi-block writes and contiguous allocation. When a write expands a file beyond its current allocation, the file system allocates a new contiguous region, copies the existing data, frees the old region, and writes the updated buffer.

```

oldBlocks = blocksForFileSize(fcb->entry.size, vcb.blockSize);
newBlocks = blocksForFileSize(newSize, vcb.blockSize);

if (newBlocks > oldBlocks)
{
    fs_allocateBlocks(newBlocks, &newStart);
    fs_readBlocks(tempBuffer, oldBlocks, fcb->entry.location);
    fs_writeBlocks(tempBuffer, newBlocks, newStart);
    fs_freeBlocks(fcb->entry.location, oldBlocks);
}

```

```

    fcb->entry.location = newStart;
}

```

Seek limitation. The system does not support sparse files, so `b_seek` rejects offsets beyond the current file size.

```

if ((uint64_t)newOffset > fcbArray[fd].entry.size)
{
    return -1;
}

```

Deletion. Deleting a file requires both clearing the directory entry and returning its blocks to the bitmap.

```

blocksUsed = (entry.size + vcb.blockSize - 1) / vcb.blockSize;
fs_freeBlocks(entry.location, blocksUsed);
memset(&entry, 0, sizeof(DirectoryEntry));

```

7. Shell Integration

The shell layer exposes the completed backend functions through the project command interface.

Command flags are enabled once the relevant backend functionality is implemented. The `mv` command is connected to `fs_move` so files and directories can be renamed or moved through the shell.

Shell / Build Area	Implementation Result
Command flags	Enabled only after backend functions are available.
<code>cmd_mv</code>	Implemented by connecting shell input to <code>fs_move</code> .
<code>ls</code> , <code>md</code> , <code>rm</code>	Use directory iteration, directory creation, directory validation, file deletion, and removal logic.
<code>touch</code> , <code>cat</code> , <code>cp</code> , <code>cp2fs</code> , <code>cp2l</code>	Use file creation, buffered read/write, and Linux-to-file-system or file-system-to-Linux copy logic.
<code>cd</code> , <code>pwd</code>	Use current working directory tracking and path traversal.
Build compatibility	The Makefile handles architecture differences by selecting <code>fsLowM1.o</code> for ARM/aarch64 systems and <code>fsLow.o</code> for x86_64 systems.

8. Plan and Implementation Approach

The implementation was divided by subsystem. The core layout work established persistent metadata and free-space management first. Directory traversal and current-working-directory support were then built on top of the layout. File operations used the directory helpers to become path-aware, and the shell was enabled once the backend functions were ready.

1. Format and identify the volume by writing a VCB with the MFSVCB1 magic number.
2. Initialize the free-space bitmap and mark the VCB, bitmap, and root directory blocks as allocated.
3. Allocate the root directory through the free-space system instead of hardcoding its storage.
4. Initialize the root directory with . and .. entries that point back to the root.
5. Implement path-aware directory operations and reusable lookup helpers.
6. Implement buffered file I/O and file metadata updates.
7. Integrate the shell commands, verify behavior, collect screenshots, and prepare the final report.

9. Issues Faced and Resolutions

Issue	Resolution
Low-level partition header vs. logical block 0	The first visible bytes in the raw volume dump were initially confusing because the low-level layer adds its own header. The team resolved this by reviewing the low-level block behavior and then reading the correct logical block containing the VCB.
Root directory allocation	The root directory was changed to request blocks from the free-space allocator instead of relying on a hardcoded location.
. and .. initialization	The root directory entries for . and .. were initialized to point to the root so traversal rules work consistently.
b_open complexity	Opening a file required resolving paths, distinguishing files from directories,

Issue	Resolution
	creating missing files under O_CREAT, and loading metadata into the FCB.
O_TRUNC behavior	Truncation releases allocated blocks and updates metadata, preventing leaked space and inconsistent directory entries.
O_APPEND behavior	Append mode sets the write offset to the current file size before writing so old contents are not overwritten.
Writes across block boundaries	The implementation calculates the new file size, expands contiguous allocation when necessary, copies old data into the new allocation, and writes the updated buffer.
Path-aware file helpers	fs_isFile, fs_stat, and fs_delete were updated to use fs_resolvePath and fs_resolveParentPath so they work inside subdirectories rather than only in the root.
Sparse files	b_seek is intentionally limited to offsets within the current file size because sparse files are not supported.

10. Team Workflow

The team used subsystem ownership to reduce conflicts and make integration easier. Each person was responsible for a specific part of the file system, while still coordinating with the rest of the group when shared functionality was needed. The core filesystem owner finalized the on-disk layout, including the VCB, permanent DirectoryEntry format, root directory structure, and free-space helpers. The directory owner implemented path traversal, current working directory behavior, and the directory APIs. The file owner implemented buffered I/O, file creation and deletion behavior, open flag handling, and file metadata updates. The shell and testing owner validated the user-facing commands, collected testing evidence, and prepared the final report materials.

The work required overlap between modules because the file system components depended on each other. Directory and file operations both depended on the permanent on-disk **DirectoryEntry** format and the free-space helper functions. File operations also depended on path resolution, while shell testing depended on all backend features being correctly integrated. One challenge was that the team was difficult to organize at first because it was formed from two half teams. After Milestone 1, we effectively had two separate Milestone 1 versions and had to decide which implementation to build on top of for the final project. This caused early communication and scheduling delays, but the team resolved the issue by dividing ownership clearly and coordinating integration work more directly. After that, we held regular weekly Zoom meetings, along with some library meetings, to review progress, discuss design decisions, debug issues, and make sure each subsystem worked with the others.

11. Shell Command Status and Limitations

Command	Status	What Works	Notes / Limitations
ls	Working	Lists files and directories; ls -l displays type and size information.	Depends on directory iteration state.
cd	Working	Changes current directory using absolute and relative paths.	Invalid target paths are rejected.
pwd	Working	Prints the full current working directory path.	Root prints as /.
md	Working	Creates a new directory and links it to its parent.	Duplicate names are rejected.
touch	Working	Creates an empty file.	Uses file creation logic through path resolution.
cat	Working	Displays file contents.	Reports an error for missing or invalid file targets.
cp	Working	Copies a file within the test file system.	Uses buffered read/write behavior.
cp2fs	Working	Copies a file from Linux into the test file system.	Verified through the screenshot set and file listing/cat output.
cp2l	Working	Copies a file from the test file system to Linux.	Shown as enabled/available in the command status and help output.
rm	Working	Removes files and directories.	Directories must be empty before removal; file deletion frees allocated blocks.

Command	Status	What Works	Notes / Limitations
mv	Working	Renames or moves files/directories.	Does not overwrite existing targets and prevents moving a directory into itself or its descendants.

12. Detailed Function-Level Contribution Table

The final division of work was organized by subsystem ownership. The table below records the function-level responsibilities and the supporting integration work.

Team Member	Primary Ownership	Primary Files / Areas
Ahmed Rashad	Filesystem Core / On-Disk Layout	fsInit.c and shared internal metadata definitions
Edson Sanchez Bernal	Directory System / Path Resolution	mfs.h, fsDir.c, fdDir updates, and shared path lookup helpers
Fnu Atiksha	File Operations / Buffered I/O	b_io.c and fsFile.c file helpers
Junxi Lin	Shell Integration / Testing / Report Readiness	fsshell.c, command validation, screenshots, and final submission materials

12.1 Function-Level Contribution Details

Team Member	Function / Component	Work Completed
Ahmed Rashad	VCB structure	Finalized persistent VCB fields, including magic number, version, total blocks, block size, bitmap location, root directory location, and root entry count.
Ahmed Rashad	Root directory layout	Defined the initial root directory layout and ensured the root directory is allocated through the free-space system.
Ahmed Rashad	DirectoryEntry / on-disk metadata	Defined a permanent on-disk metadata format and avoided runtime pointers in persistent structures.
Ahmed Rashad	Free-space helpers	Implemented reusable block allocation, block freeing, mark-used/mark-free behavior, and bitmap persistence.
Ahmed Rashad	Mount/unmount persistence	Ensured VCB, bitmap, root directory, and metadata blocks are read and written consistently across executions.
Ahmed Rashad	Directory/metadata block helpers	Provided helper functions other modules use to read and write directory blocks and metadata blocks.
Ahmed Rashad	Core comments/headers	Cleaned up comments and headers in core filesystem files.
Edson Sanchez Bernal	fdDir in mfs.h	Updated fdDir so it tracks loaded entries, current position, directory location,

Team Member	Function / Component	Work Completed
		block count, and fs_direntinfo output.
Edson Sanchez Bernal	fs_resolvePath	Implemented reusable resolution for absolute paths, relative paths, ., and ...
Edson Sanchez Bernal	fs_resolveParentPath	Implemented parent lookup for create/delete/move operations and shared it with file I/O.
Edson Sanchez Bernal	fs_mkdir	Created directories, rejected duplicate names, initialized . and .., and linked the child to its parent.
Edson Sanchez Bernal	fs_rmdir	Validated targets and removed directories only when they are empty.
Edson Sanchez Bernal	fs_opendir / fs_readdir / fs_closedir	Implemented directory opening, iteration, and cleanup for shell listing operations.
Edson Sanchez Bernal	fs_getcwd / fs_setcwd	Implemented current working directory tracking and validated directory changes.
Edson Sanchez Bernal	fs_isDir	Implemented directory-type validation for resolved paths.
Edson Sanchez Bernal	fs_move support	Added move/rename validation, including no overwrite and no moving a directory into itself or its children.
Edson Sanchez Bernal	Build compatibility	Updated the Makefile to select the correct low-level object file for ARM/aarch64 or x86_64 Ubuntu VMs.

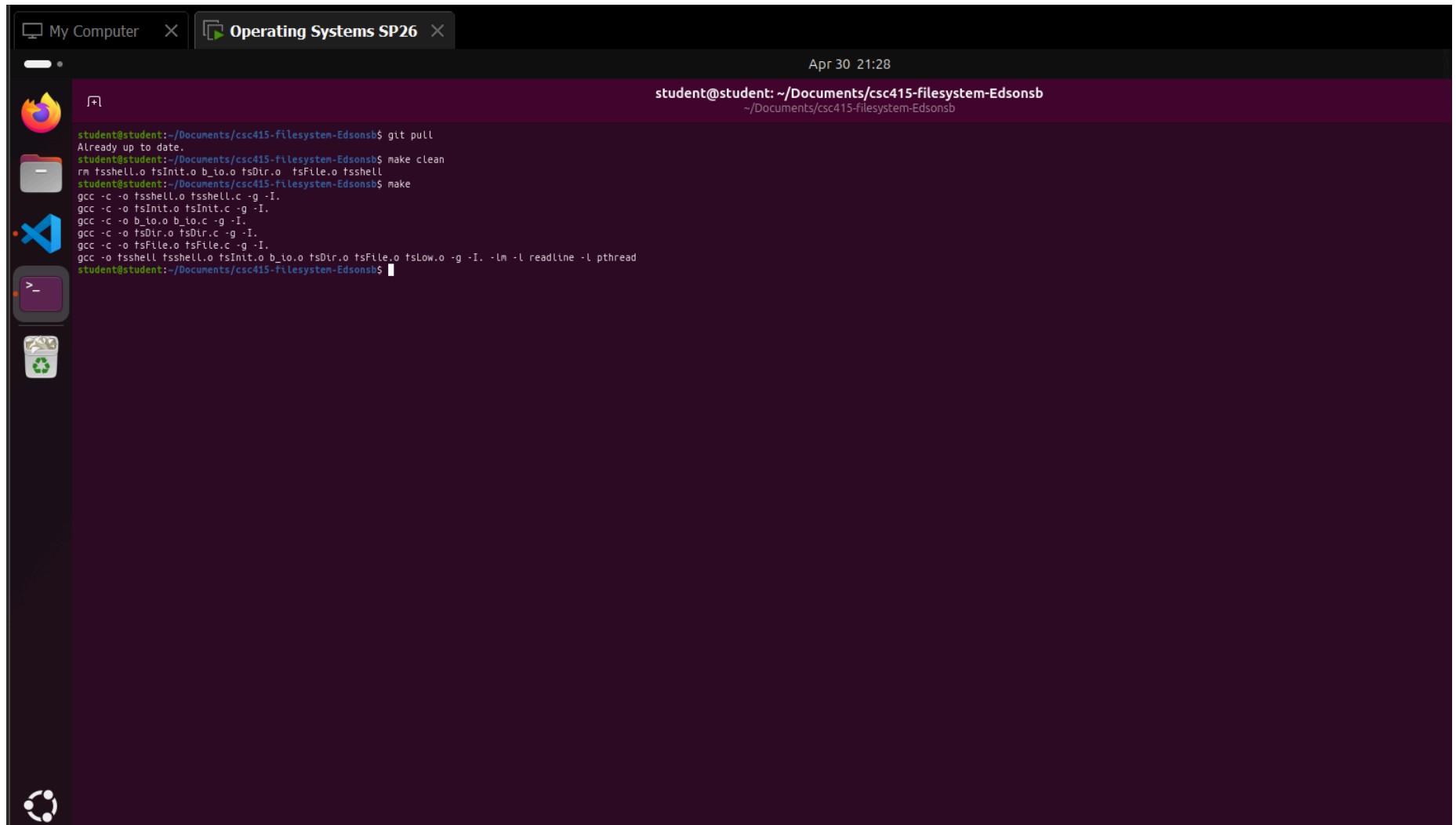
Team Member	Function / Component	Work Completed
Junxi Lin	FCB table in b_io.c	Expanded the FCB structure to store DirectoryEntry metadata, offset, access flags, parent directory location, and entry index.
Junxi Lin	b_open	Resolved files, handled file-vs-directory validation, created files under O_CREAT, handled O_TRUNC and O_APPEND, and populated an FCB slot.
Junxi Lin	b_read	Read file data from the current offset, respected EOF behavior, and advanced the file offset.
Junxi Lin	b_write	Handled writes across block boundaries, expanded allocation when needed, updated file size and metadata, and advanced the offset.
Junxi Lin	b_seek	Implemented offset movement within the current file size and rejected sparse-file-style seeks beyond EOF.
Junxi Lin	b_close	Persisted file metadata updates and released the FCB slot.
Junxi Lin	Open flags	Supported O_CREAT, O_TRUNC, O_APPEND, O_RDONLY, O_WRONLY, and O_RDWR.
Junxi Lin	fs_isFile / fs_delete / fs_stat	Implemented path-aware file validation, deletion

Team Member	Function / Component	Work Completed
		with block release, and metadata reporting.
Junxi Lin	File metadata updates	Updated file size, allocated block information, and access/modify/create time behavior.
Fnu Atiksha	fsshell.c command flags	Enabled shell commands when their backend functions were complete.
Fnu Atiksha	cmd_mv	Integrated mv command behavior with fs_move.
Fnu Atiksha	Shell command validation	Verified ls, md, rm, touch, cat, cp, cp2l, cp2fs, cd, pwd, and mv behavior.
Fnu Atiksha	Screenshots and report evidence	Collected command screenshots and organized the works/limitations evidence table.
Fnu Atiksha	Final write-up readiness	Cleaned user-facing documentation, command notes, and report sections describing how the implemented functions work.

13. Screenshots Showing Required Commands

The screenshots below preserve the command evidence from the original document. Together they show build/startup, enabled command flags, directory commands, file commands, copy commands, move/rename behavior, removal behavior, and error handling.

Command Evidence	Screenshot Location
Build/startup	Figures 5 and 6
ls, pwd, help	Figure 6
md, cd, pwd, directory traversal	Figure 7
touch, ls -l, cp2fs, cat, cp	Figure 8
cat, cd, mv, help output	Figure 9
rm	Figure 10
Error handling for invalid cd/rm/mv targets	Figure 11
cp2l availability	Figure 11 shows copying a file from the test file system back to the Linux file system.

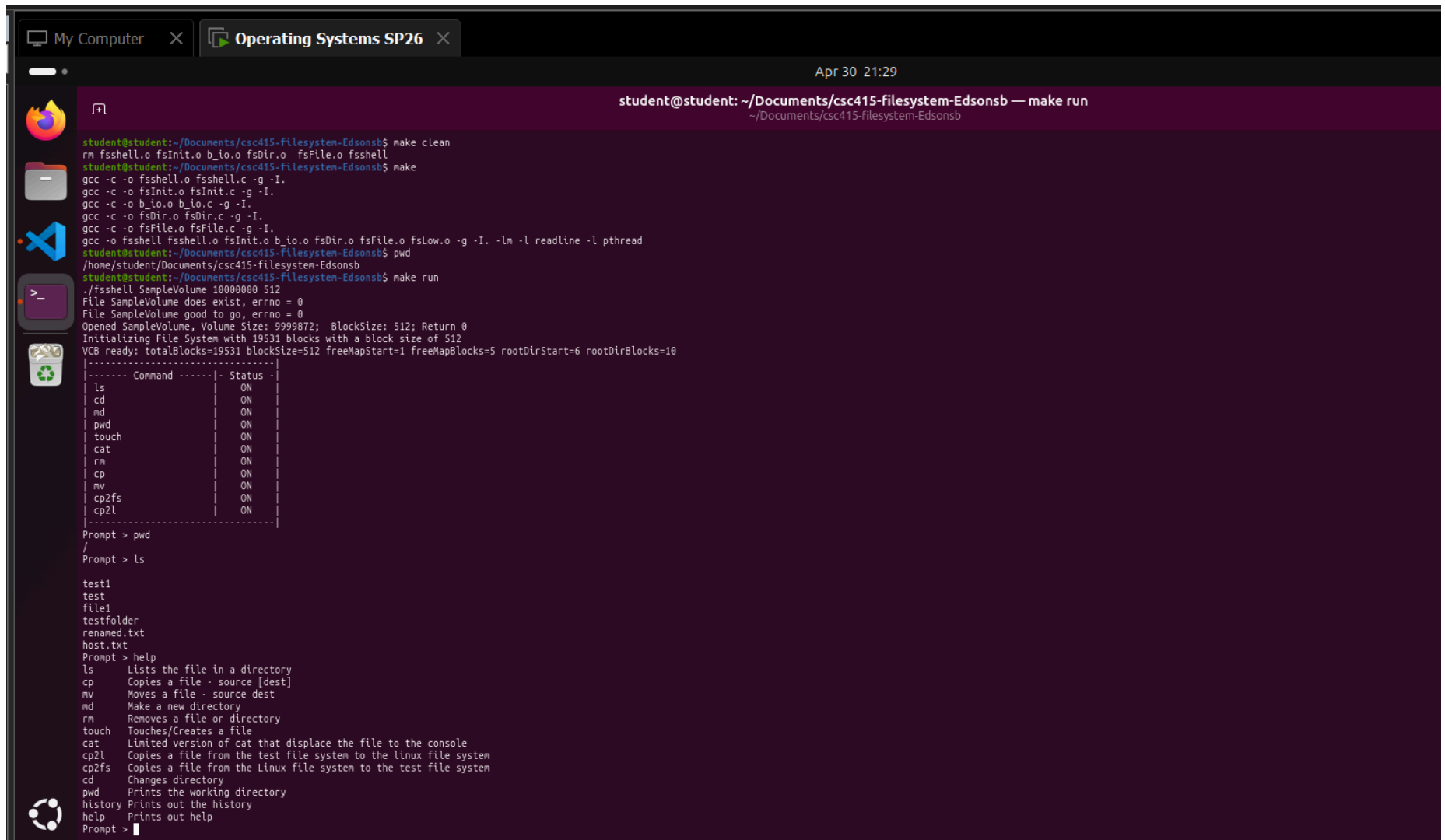


The image shows a terminal window titled "Operating Systems SP26" with a dark theme. The terminal displays the following commands and output:

```
student@student:~/Documents/csc415-filesystem-Edsonsb$ git pull
Already up to date.
student@student:~/Documents/csc415-filesystem-Edsonsb$ make clean
rm fsshell.o tsInit.o b_io.o tsDir.o tsFile.o fsshell
student@student:~/Documents/csc415-filesystem-Edsonsb$ make
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o tsInit.o tsInit.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -c -o tsDir.o tsDir.c -g -I.
gcc -c -o tsFile.o tsFile.c -g -I.
gcc -o fsshell fsshell.o tsInit.o b_io.o tsDir.o tsFile.o tsLow.o -g -I. -lm -l readline -l pthread
student@student:~/Documents/csc415-filesystem-Edsonsb$
```

The terminal window includes a sidebar with icons for a file manager, a code editor, and a terminal. The top bar shows the window title and the current date and time (Apr 30 21:28).

Figure 5. Build verification after git pull, make clean, and make.

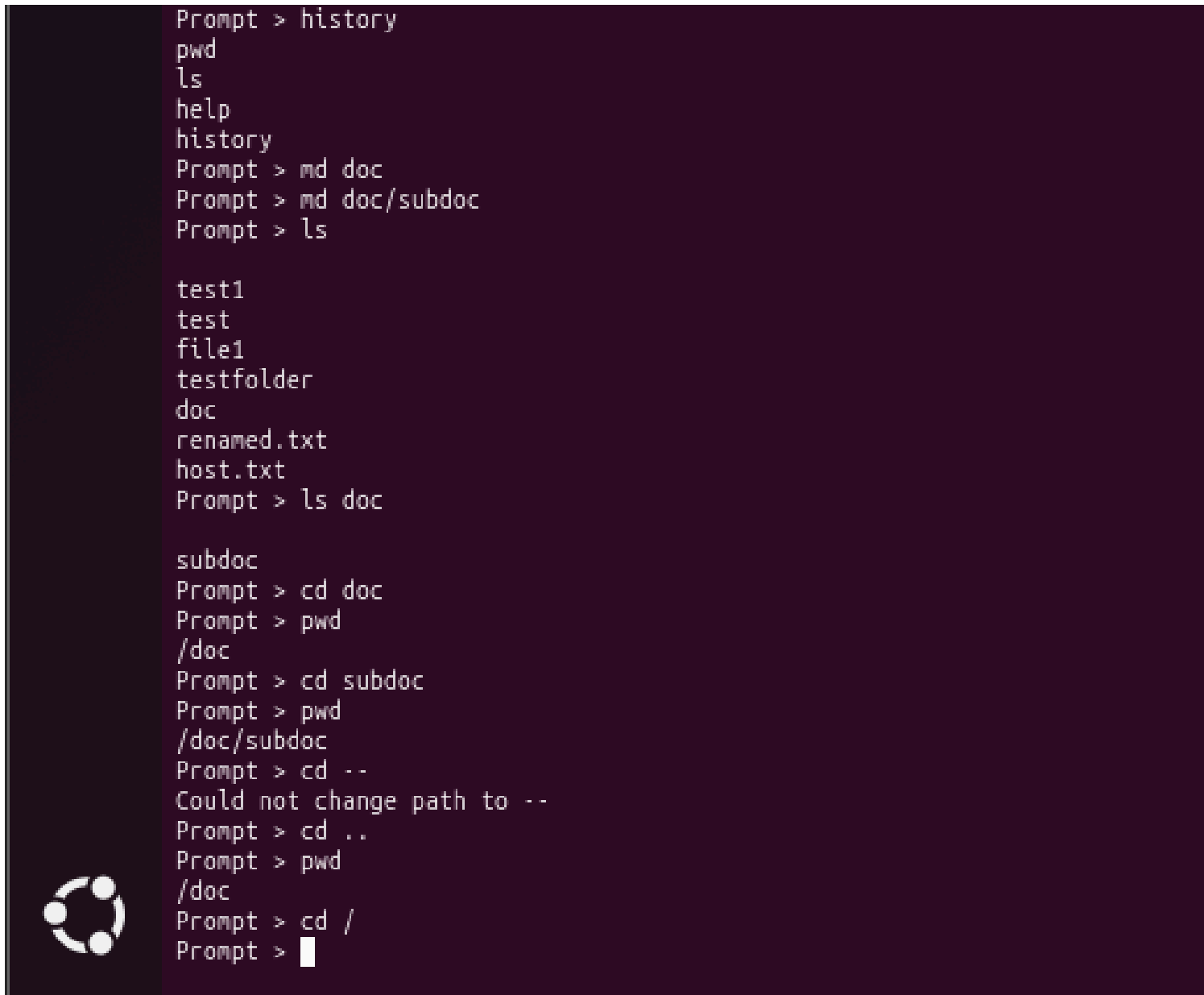


```
student@student: ~/Documents/csc415-filesystem-Edsonsb — make run
~/Documents/csc415-filesystem-Edsonsb

student@student:~/Documents/csc415-filesystem-Edsonsb$ make clean
rm fsshell.o fsInit.o b_io.o fsDir.o fsFile.o fsshell
student@student:~/Documents/csc415-filesystem-Edsonsb$ make
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -c -o fsDir.o fsDir.c -g -I.
gcc -c -o fsFile.o fsFile.c -g -I.
gcc -o fsshell fsshell.o fsInit.o b_io.o fsDir.o fsFile.o fsLow.o -g -I. -lm -l readline -l pthread
student@student:~/Documents/csc415-filesystem-Edsonsb$ pwd
/home/student/Documents/csc415-filesystem-Edsonsb
student@student:~/Documents/csc415-filesystem-Edsonsb$ make run
./fsshell SampleVolume 100000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872; BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
VCB ready: totalBlocks=19531 blockSize=512 freeMapStart=1 freeMapBlocks=5 rootDirStart=6 rootDirBlocks=10
----- Command -----| Status |
ls                        | ON    |
cd                        | ON    |
md                        | ON    |
pwd                       | ON    |
touch                    | ON    |
cat                       | ON    |
rm                        | ON    |
cp                        | ON    |
mv                        | ON    |
cp2fs                     | ON    |
cp2l                      | ON    |
-----|-----|
Prompt > pwd
/
Prompt > ls

test1
test
file1
testfolder
renamed.txt
host.txt
Prompt > help
ls      Lists the file in a directory
cp      Copies a file - source [dest]
mv      Moves a file - source dest
md      Make a new directory
rm      Removes a file or directory
touch   Touches/Creates a file
cat     Limited version of cat that displace the file to the console
cp2l    Copies a file from the test file system to the linux file system
cp2fs   Copies a file from the Linux file system to the test file system
cd      Changes directory
pwd     Prints the working directory
history Prints out the history
help    Prints out help
Prompt > |
```

Figure 6. make run startup, VCB initialization output, command status table, pwd, ls, and help output.

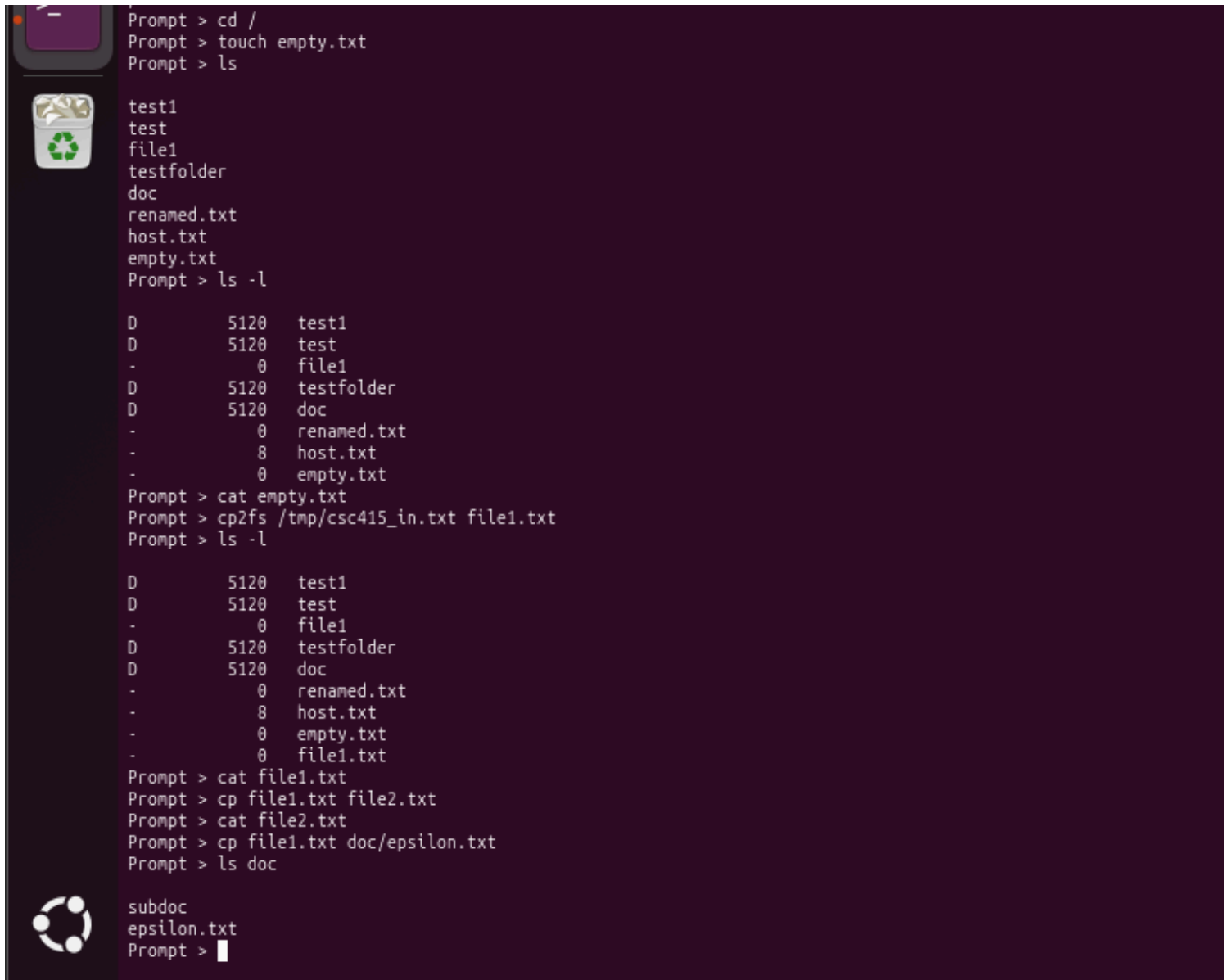
A terminal window with a dark purple background. On the left side, there is a vertical bar with the Ubuntu logo (a white circle with four dots) at the bottom. The terminal text shows a sequence of commands and their outputs. The commands include 'history', 'md doc', 'md doc/subdoc', 'ls', 'cd doc', 'cd subdoc', 'cd ..', and 'cd /'. The outputs show the current directory path and a list of files in the current directory.

```
Prompt > history
pwd
ls
help
history
Prompt > md doc
Prompt > md doc/subdoc
Prompt > ls

test1
test
file1
testfolder
doc
renamed.txt
host.txt
Prompt > ls doc

subdoc
Prompt > cd doc
Prompt > pwd
/doc
Prompt > cd subdoc
Prompt > pwd
/doc/subdoc
Prompt > cd --
Could not change path to --
Prompt > cd ..
Prompt > pwd
/doc
Prompt > cd /
Prompt > 
```

Figure 7. Directory command evidence: md, nested directory creation, ls, cd, pwd, cd .., and cd /.

A terminal window with a dark purple background. On the left side, there are three icons: a purple folder icon, a trash can icon, and a circular icon with three dots. The terminal shows a series of commands and their outputs. The commands include 'cd /', 'touch empty.txt', 'ls', 'ls -l', 'cat empty.txt', 'cp2fs /tmp/csc415_in.txt file1.txt', 'ls -l', 'cat file1.txt', 'cp file1.txt file2.txt', 'cat file2.txt', 'cp file1.txt doc/epsilon.txt', 'ls doc', and a final prompt. The outputs show the directory listing of the root directory and the 'doc' subdirectory.

```
Prompt > cd /
Prompt > touch empty.txt
Prompt > ls

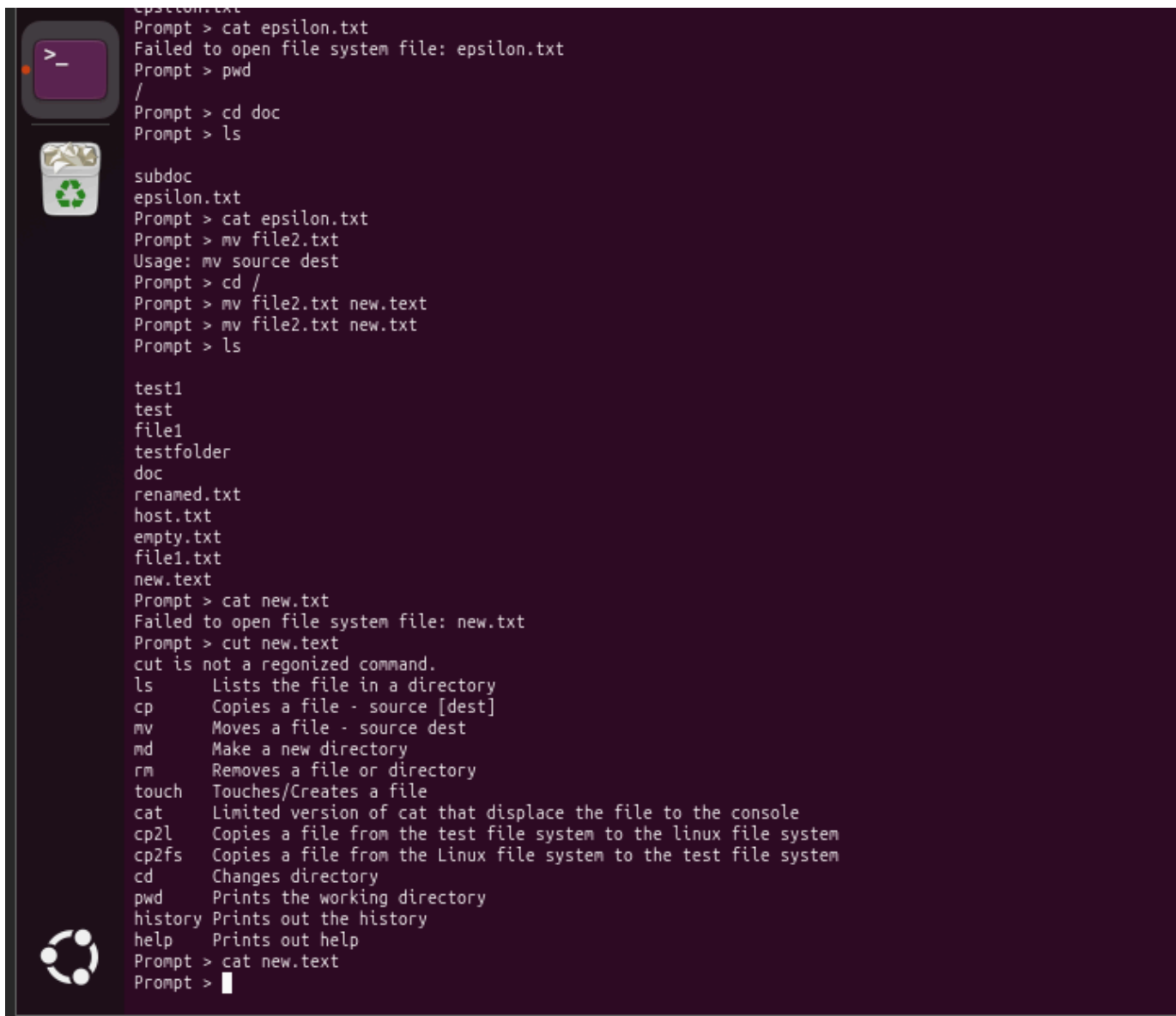
test1
test
file1
testfolder
doc
renamed.txt
host.txt
empty.txt
Prompt > ls -l

D          5120  test1
D          5120  test
-           0    file1
D          5120  testfolder
D          5120  doc
-           0    renamed.txt
-           8    host.txt
-           0    empty.txt
Prompt > cat empty.txt
Prompt > cp2fs /tmp/csc415_in.txt file1.txt
Prompt > ls -l

D          5120  test1
D          5120  test
-           0    file1
D          5120  testfolder
D          5120  doc
-           0    renamed.txt
-           8    host.txt
-           0    empty.txt
-           0    file1.txt
Prompt > cat file1.txt
Prompt > cp file1.txt file2.txt
Prompt > cat file2.txt
Prompt > cp file1.txt doc/epsilon.txt
Prompt > ls doc

subdoc
epsilon.txt
Prompt > 
```

Figure 8. File command evidence: touch, ls, ls -l, cp2fs, cat, cp, and copying into a subdirectory.

A terminal window with a dark purple background. On the left side, there are three icons: a terminal icon (a purple square with a white prompt character), a trash can icon (a white trash can with a green recycling symbol), and a circular arrow icon (a white circular arrow). The terminal text shows a series of commands and their outputs. The first command is 'cat epsilon.txt', which results in an error: 'Failed to open file system file: epsilon.txt'. Subsequent commands include 'pwd' (output: '/'), 'cd doc', and 'ls' (output: 'subdoc', 'epsilon.txt'). Then, 'cat epsilon.txt' is run again. This is followed by 'mv file2.txt', which shows usage information. Then 'cd /' is run, followed by 'mv file2.txt new.txt' and another 'mv file2.txt new.txt'. Then 'ls' is run, listing various files and directories. Next, 'cat new.txt' is run, resulting in an error: 'Failed to open file system file: new.txt'. Then 'cut new.txt' is run, resulting in an error: 'cut is not a recognized command.'. Finally, a help message is displayed, listing various commands and their functions. The last command is 'cat new.txt', which is followed by a prompt character.

```
epsilon.txt
Prompt > cat epsilon.txt
Failed to open file system file: epsilon.txt
Prompt > pwd
/
Prompt > cd doc
Prompt > ls

subdoc
epsilon.txt
Prompt > cat epsilon.txt
Prompt > mv file2.txt
Usage: mv source dest
Prompt > cd /
Prompt > mv file2.txt new.txt
Prompt > mv file2.txt new.txt
Prompt > ls

test1
test
file1
testfolder
doc
renamed.txt
host.txt
empty.txt
file1.txt
new.txt
Prompt > cat new.txt
Failed to open file system file: new.txt
Prompt > cut new.txt
cut is not a recognized command.
ls      Lists the file in a directory
cp      Copies a file - source [dest]
mv      Moves a file - source dest
md      Make a new directory
rm      Removes a file or directory
touch   Touches/Creates a file
cat     Limited version of cat that displace the file to the console
cp2l    Copies a file from the test file system to the linux file system
cp2fs   Copies a file from the Linux file system to the test file system
cd      Changes directory
pwd     Prints the working directory
history Prints out the history
help    Prints out help
Prompt > cat new.txt
Prompt > █
```

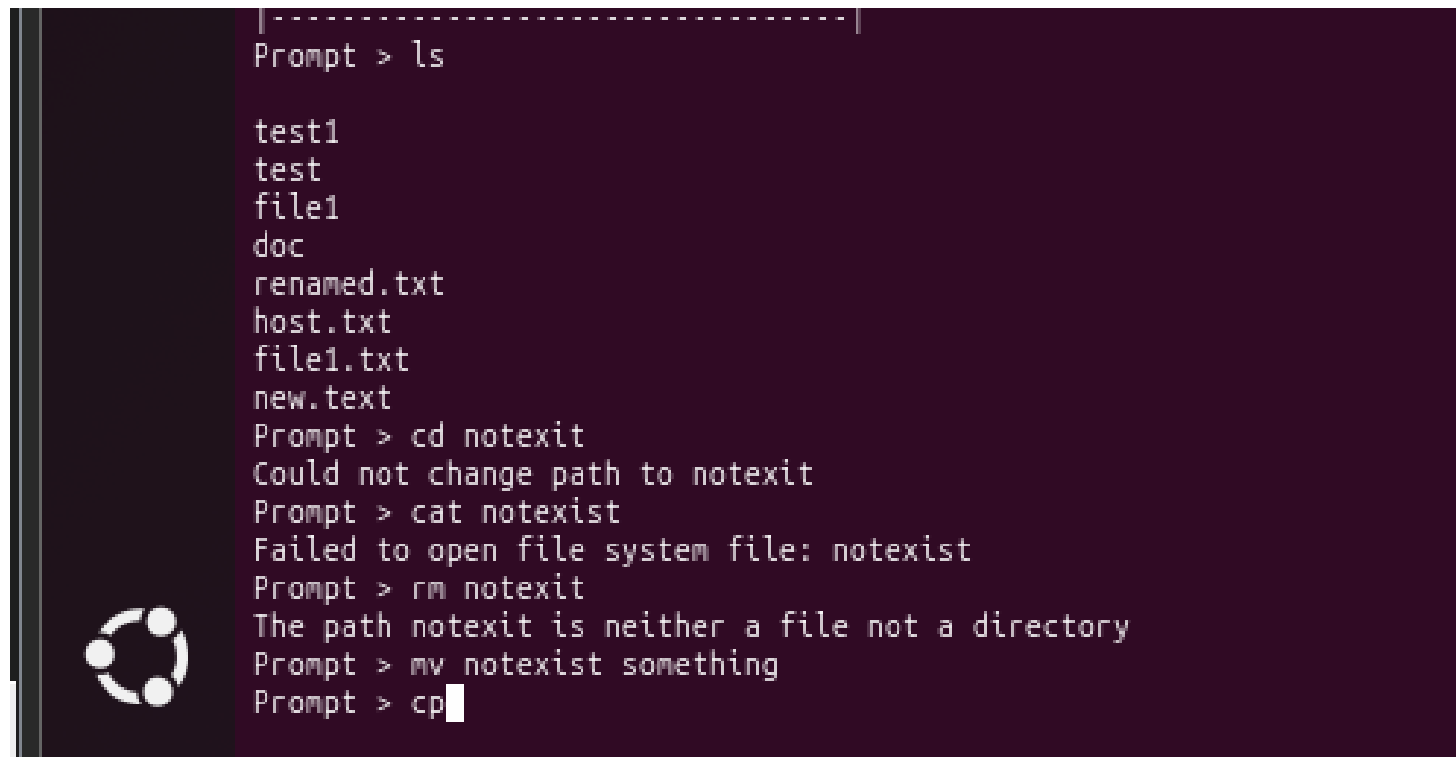
Figure 9. File and shell command evidence: cat, pwd, cd, mv/rename behavior, ls, help, and error output for missing files.

```
Prompt > rm doc
Prompt > ls

test1
test
file1
testfolder
doc
renamed.txt
host.txt
file1.txt
new.txt
Prompt > rm testfolder
Prompt > ls

test1
test
file1
doc
renamed.txt
host.txt
file1.txt
new.txt
Prompt > S
```

Figure 10. Removal command evidence: `rm testfolder` followed by `ls` showing the directory removed.

A terminal window with a dark purple background and a light gray border. On the left side, there is a vertical bar with the Ubuntu logo (a white circle with four dots) and a small white square. The terminal text is as follows:

```
-----  
Prompt > ls  
  
test1  
test  
file1  
doc  
renamed.txt  
host.txt  
file1.txt  
new.text  
Prompt > cd notexit  
Could not change path to notexit  
Prompt > cat notexist  
Failed to open file system file: notexist  
Prompt > rm notexit  
The path notexit is neither a file not a directory  
Prompt > mv notexist something  
Prompt > cp
```

Figure 11. Error-handling evidence for invalid `cd`, `cat`, `rm`, and `mv` targets.